DocMind to Production

System Design Notes — 1

Scale from a single laptop to 2000 users on AWS

Goal: To understand how software runs in production, why it breaks under load, and how to design systems that don’t. These notes supplement your YouTube videos — every concept here maps directly to DocMind.

Part 1: How Code Gets Hosted — The Absolute Basics

1.1 Your Laptop vs The Internet

Right now, when you run DocMind with `uvicorn main:app --reload`, it starts a web server on your machine. Open a browser, go to `http://localhost:8000`, and your browser sends a request to that server and gets a response back. All of this happens inside your laptop.

The moment you close your laptop — or your wifi drops — DocMind stops working for everyone. Nobody else on the internet can reach it. That is the fundamental problem you solve by deploying to a cloud service like AWS.

> **The Chai Stall Analogy**
>
> Your laptop is a chai stall you run in your bedroom. Only people physically in your bedroom can order.
>
> AWS is a chai stall on a busy commercial street — always open, always reachable, scales when demand grows.

What actually happens when a user makes a request?

Every interaction with your app triggers a chain of events called the **request-response cycle**:

- User types a URL or presses a button in their browser
- Their browser converts that URL to an IP address (via DNS — like a phone book for websites)
- It sends an HTTP request to that IP address
- Your server receives it, processes it, and sends back an HTTP response
- The browser renders the response and shows the user their answer

Everything in system design is about making this cycle fast, reliable, and scalable under real-world conditions.

1.2 What is a Server?

A server is just a computer that listens for incoming requests and responds to them. It runs 24/7, has a stable IP address, and has more RAM and CPU than a typical laptop.

Type	Characteristics
Your laptop	Turns off. No fixed IP. Not accessible to the internet. Only one person at a time.
A dedicated server	Always on. Fixed IP. Accessible globally. Multiple people simultaneously.
Cloud server (AWS)	Same as above, but scales up/down automatically. You pay only for what you use.

1.3 HTTP and APIs — The Language of the Web

Your FastAPI application speaks HTTP — HyperText Transfer Protocol. Think of it as the agreed-upon language that browsers and servers use to communicate. Every request has:

- **Method:** GET (fetch data), POST (send data), PUT (update), DELETE (remove)
- **URL path:** e.g. `/upload` or `/query`
- **Headers:** metadata about the request (content type, authentication token)
- **Body:** the actual data being sent (e.g., the PDF file, or the question text)

When you write this in FastAPI:

```
@app.post("/query")async def query_document(request: QueryRequest):
...
```

You're telling the server: "when a POST request arrives at /query, run this function." That's all an API is — a contract for how to communicate with your application.

Part 2: Why Systems Break Under Load

2.1 The Single Chai Guy Problem

A chai stall run by one person has a simple workflow: take order → make chai (2 min) → hand over → next customer. With one customer every 3 minutes, perfect. With 15 customers arriving simultaneously, the 15th person waits 30 minutes for a 2-minute job.

This is your DocMind system today. Every request blocks everything else.

> **DocMind today — the chain of blocking operations**
>
> User uploads PDF → app reads it → chunks it → calls OpenAI to embed → stores in Pinecone → returns done.
>
> Every step blocks the next user. If embedding takes 10s, User 2 waits idle for those 10s before being processed.

2.2 Synchronous vs Asynchronous Code — The Most Important Concept Today

Synchronous (blocking) — the bad default

Synchronous code does one thing at a time and waits for each step to finish before starting the next:

```
# Synchronous — BLOCKS everything while waiting for external callsdef query(question):    chunks = retrieve_from_db(question)    # waits 200ms doing nothing    answer = call_openai(chunks)           # waits 2000ms doing nothing    return answer                           # total: 2200ms blocked
```

If 15 users send queries simultaneously: the 15th user waits **15 × 2200ms = 33 seconds** just because of queuing. The server is not busy — it is just sitting idle waiting for API responses.

Asynchronous (non-blocking) — what you need

Async code fires off a task and says “tell me when you’re done.” Meanwhile it handles the next request. Like a waiter who takes multiple orders, gives them all to the kitchen simultaneously, and delivers each when ready — not one at a time.

```
# Async — non-blocking, many users handled concurrentlyasync def query(question):    chunks = await retrieve_from_db(question)    # pauses HERE, serves User 2    answer = await call_openai(chunks)           # pauses HERE, serves User 3    return answer
# resumes when OpenAI responds
```

With async code, while User 1’s OpenAI call is waiting for a response, the server starts processing Users 2, 3, 4... simultaneously. **One server can handle 15+ concurrent users** instead of 1.

> **Key Insight**
>
> The await keyword is the difference between a server handling 1 user at a time and 50. FastAPI is built on asyncio and supports this natively. Converting your routes to async def with await on all I/O calls is the single highest-impact change you make to DocMind.

2.3 The Three Bottlenecks in Every System

Bottleneck	What it is	DocMind impact	Fix
CPU	Server runs out of processing power	Low — RAG is not CPU-heavy	Add more servers
Network I/O	Server waits idle for external responses	HIGH — every query waits on OpenAI + Pinecone	Async code + caching
Storage	DB can't handle concurrent reads/writes	Medium — relevant at thousands of users	Connection pooling, managed vector DB

For DocMind at 2000 users, **Network I/O is your primary bottleneck.** Every query waits on OpenAI and Pinecone. Async code directly solves this.

Part 3: Scaling — From One to Two Thousand Users

3.1 Vertical vs Horizontal Scaling

Type	What it means	Pros	Cons
Vertical Scaling (Scale Up)	Buy a bigger, faster server. More CPU, more RAM.	Simple. No code changes.	Expensive. Has a ceiling. Single point of failure — one crash = everything down.
Horizontal Scaling (Scale Out)	Add more servers running the same code. Multiple identical servers behind a load balancer.	Unlimited scale. If one server fails, others keep running. Cost-efficient.	Requires stateless design. More infrastructure to manage.

> **The real answer to the panel's question**
>
> When they asked "how would you scale to 2000 users?" they were expecting horizontal scaling as the answer.
>
> Not "I'd get a bigger server." But: "I'd containerise the app, deploy multiple stateless instances on ECS, put an ALB in front, and add a cache layer to reduce LLM calls."

3.2 Stateless Design — The Prerequisite for Horizontal Scaling

For horizontal scaling to work, each server must be **stateless** — it must not remember anything about previous requests. Every request carries everything the server needs to process it.

Why? If User 1's data is stored in Server A's memory, and their next request goes to Server B, Server B has no idea who they are. The system breaks.

> **McDonald's Analogy**
>
> A McDonald's drive-through is stateless. You give your full order every time. They don't remember you. Any window serves you identically.
>
> This is why McDonald's scales to thousands of locations. Your DocMind needs the same property.

What stateless means for DocMind specifically:

- Do not store uploaded files locally on the server — store them in S3
- Do not store user sessions in server memory — use JWTs or a shared session store
- Do not use global variables to store request state — if the server restarts, it starts clean
- Every request contains its own context — user ID, document ID, etc.

3.3 Load Balancers — The Traffic Director

Once you have multiple servers, something must sit in front of them and decide which server handles each request. That's a **load balancer**.

> **Hospital receptionist analogy**
> Patients arrive at a busy hospital. The receptionist looks at which doctor is least busy and sends each patient to that doctor.
> No single doctor gets overwhelmed while others sit idle. The load is balanced.

Strategy	How it works	Best for
Round Robin	Request 1 → Server A, Request 2 → Server B, Request 3 → Server C, back to A...	Simple workloads where all requests are similar in processing time
Least Connections	Route each request to whichever server currently has fewest active connections	RAG workloads — some queries take 3s, others take 200ms
IP Hash	Route same user's requests to the same server every time	When server-side local caching is used

In AWS, the **Application Load Balancer (ALB)** handles this automatically. You configure it once; it manages all traffic distribution across your ECS container instances.

3.4 Message Queues — Handling Spikes Without Crashing

Document ingestion in DocMind (PDF parsing + chunking + embedding) takes 10–30 seconds. Making users wait that long is terrible UX. And if 50 users upload simultaneously, your server is overwhelmed.

> **Dry cleaning analogy**
> You drop off your clothes (instant acknowledgement), they give you a ticket, and you pick up later.
> The shop does not stop accepting new clothes while they clean your suit. The work happens asynchronously in the background.

A message queue applies the same pattern to your app:

- User uploads PDF → you return "processing" instantly (< 1 second)
- Job is placed in the queue (AWS SQS or Redis)
- A separate worker process picks it up and processes it in the background
- User gets notified when chunks are ready to query

Popular queue tools: **AWS SQS** (managed, reliable), **Celery** (Python-native task queue), **Redis Queues** (fast, in-memory).

Part 4: Caching — Don’t Do the Same Work Twice

4.1 What is a Cache?

A cache stores the result of an expensive operation so you can return it instantly next time without redoing the work.

> **Teacher analogy**
>
> A teacher gets the same question from 30 students. First time: looks it up, thinks through it, answers (expensive, takes 2 minutes).
>
> Then writes the answer on the board (the cache). Students 2–30 just read the board — instant, free.
>
> For DocMind: if 100 users ask the same question about the same document, why call OpenAI 100 times?

4.2 The Four Caching Strategies

1. Cache Aside (most common — use this in DocMind)

The application checks the cache first. If found (cache hit), return it. If not (cache miss), compute the answer, store it, return it.

```
async def query(question, doc_id):    cache_key =
f"{doc_id}:{hash(question)}"    # Step 1: check cache    cached = await
cache.get(cache_key)    if cached:        return cached   # instant! no
LLM call.    # Step 2: cache miss — do real work    chunks = await
retrieve(question, doc_id)    answer = await call_llm(chunks, question)
# Step 3: store for next time (expire after 1 hour)    await
cache.set(cache_key, answer, ttl=3600)    return answer
```

2. Semantic Caching (RAG-specific — mention this in the interview)

Regular caching matches exact strings. Semantic caching matches **meaning**. The questions “What are the contract terms?” and “Tell me about the contract conditions” are different strings but mean the same thing.

Tools like **GPTCache** embed the incoming question, check cosine similarity against previously asked questions (threshold ~0.95), and return the cached answer if it’s close enough. This can reduce LLM calls by 40–60% in practice.

3. Write-Through Cache

Every time you write data to your database, simultaneously write it to the cache. Cache is always fresh. Good for data that is frequently read and regularly updated.

4. TTL (Time-to-Live) — Cache Expiry

Every cached item has an expiry time. After the TTL, the item is removed and the next request recomputes it. Critical for RAG: if the underlying document changes, you do not want stale answers being returned.

Rule of thumb for DocMind: cache query answers for 1 hour. Cache embeddings for 24 hours. Invalidate document cache when a document is re-uploaded.

4.3 Where to Store the Cache

Cache Type	When to use	Notes
Python dict / lru_cache	Local dev only	Fast. Dies on restart. Not shared across multiple server instances.
Redis	Production standard	Fast in-memory. Persists. Shared across ALL instances — critical for horizontal scaling. AWS ElastiCache = managed Redis.
Pinecone/Vector DB	Already doing this	Semantic similarity search is inherently a form of caching. Avoids re-embedding.

> **For the interview**
>
> "I start with a simple in-memory cache for development, then move to Redis via AWS ElastiCache for production. Redis is shared across all ECS instances, so a cache hit on Server A also benefits a user on Server B."

Part 5: The Full Production Architecture for DocMind

5.1 The Evolution — Three Stages

Stage 1 — What you have now

```
User → FastAPI (your laptop) → OpenAI API                          → Pinecone                          → Local file storageProblems: synchronous blocking, laptop-dependent, 1 user at a time, no monitoring, no deployment
```

Stage 2 — Async + Caching (this week's goal)

```
User → FastAPI (async) → Cache check → return instantly if hit
→ Cache miss: Pinecone → OpenAI → cache result                          → S3 (document storage, not local disk)                          → Langfuse (trace every single request)Improvement: handles concurrent users, reduces LLM cost, observable
```

Stage 3 — Production AWS Architecture (target by April 9th)

```
2000 Users   ↓API Gateway  ← SSL termination, rate limiting, auth
↓Application Load Balancer  ← distributes traffic   ↓          ↓
↓[ECS Task 1][ECS Task 2][ECS Task 3]  ← your Docker containers (stateless)   ↓[Redis Cache]  ← check first, return if hit   ↓ (cache miss only)[Pinecone]  ← retrieve semantically similar chunks   ↓[AWS Bedrock / OpenAI]  ← generate answer   ↓[Langfuse]  ← trace logged (latency, cost, chunks used)   ↓Response to userS3  ← all PDF uploads stored hereCloudWatch  ← container CPU, memory, error ratesGitHub Actions → ECR → ECS  ← CI/CD pipeline
```

5.2 Back-of-the-Envelope Cost Calculation

The panel will ask about numbers. Here is your answer, prepared in advance:

Metric	Calculation	Result
Daily Active Users	—	2,000
Avg queries per user/day	—	10
Total queries/day	2000 × 10	20,000
Avg tokens per query (input + output)	~500 context chunks + 1500 prompt/response	~2,000 tokens
LLM cost per query (GPT-4o mini)	$0.15 per 1M input + $0.60 per 1M output	~$0.0006 per query
Monthly LLM cost (no cache)	20,000 × 30 × $0.0006	~$360/month

With 30% cache hit rate	14,000 actual LLM calls/day instead of 20,000	~$252/month
Monthly saving from cache	—	~$108/month

Interview tip — say this out loud, practise it

"At 2000 DAUs averaging 10 queries each, that's 600,000 LLM calls per month. Uncached at GPT-4o mini rates, that's around $360/month.

A 30% query cache hit rate brings that down to $252/month — saving $108. The cache pays for itself in the first few days of the month.

I know these exact numbers because I track per-query cost in Langfuse."

Part 6: Monitoring and Observability

6.1 The Four Pillars of Observability

Pillar	What it is	Tool in DocMind
Logging	Record of what happened. Plain text. “User X queried doc Y at 14:32, got answer in 1.2s.”	Python logging, CloudWatch Logs
Metrics	Numerical measurements over time. Avg response time. Requests/minute. Error rate.	CloudWatch Metrics
Tracing	Follow ONE request through the entire system. See each step’s latency.	Langfuse
Alerting	Notification when a metric crosses a threshold. “Error rate > 5% for 5 minutes → alert.”	CloudWatch Alarms

6.2 Langfuse — Your LLM Observability Layer

Langfuse is purpose-built for LLM applications. Every query generates a **trace** — a complete record of everything that happened:

- Input: the user’s original question
- Retrieval: which chunks were fetched, their relevance scores, retrieval latency
- Prompt: the exact prompt sent to the LLM (system message + context + question)
- Output: the LLM’s response
- Latency: time for each step (retrieval time, LLM time, total end-to-end)
- Cost: tokens used and exact USD cost for this query

> **Why this is your interview differentiator**
>
> No other junior candidate will be able to open a live dashboard and say:
>
> “This query cost $0.0008, retrieval took 183ms, the LLM took 1.4s, and here are the 3 chunks that were used.”
>
> Being able to show real data from real usage signals that you think like a production engineer, not a notebook user.

6.3 RAGAS — Evaluating Answer Quality

Langfuse tells you about **performance**. RAGAS tells you about **quality**. Four metrics:

RAGAS Metric	What it measures	Score	Good score to aim for
Faithfulness	Is every claim in the answer supported by the retrieved chunks? Measures hallucination.	0 to 1	Above 0.80
Answer Relevancy	Does the answer actually address what the user asked?	0 to 1	Above 0.75
Context Precision	Of the chunks retrieved, how many were actually useful?	0 to 1	Above 0.70
Context Recall	Did you retrieve all the chunks needed to fully answer the question?	0 to 1	Above 0.70

> **How to use RAGAS for the interview**
>
> Create 15 question/answer pairs from one of your sample documents.
>
> Run RAGAS. Get scores. Write one interpretive sentence per metric.
>
> Example: “Faithfulness of 0.73 means 27% of the time my system adds information not in the source. I’d address this by tightening the system prompt to instruct the model to only use retrieved context.”
>
> That interpretation + fix is what makes you stand out.

6.4 CloudWatch — AWS Infrastructure Health

While Langfuse monitors your AI pipeline, CloudWatch monitors your infrastructure. It collects metrics automatically from every AWS service:

- ECS container CPU and memory usage (if CPU > 80% consistently → add another instance)
- Number of requests hitting the ALB (shows traffic patterns, peaks)
- Application error rates — HTTP 5XX responses indicate server errors
- Lambda execution times and error counts
- Estimated AWS spend (critical for cost-aware engineering)

You do not write much code for CloudWatch — most is automatic. But in the interview you can say: *"I have Langfuse monitoring LLM pipeline quality and CloudWatch monitoring infrastructure health. Between the two, I have full-stack observability."*

Part 7: Interview Answers — Say These Out Loud

7.1 "How would you scale DocMind to 2000 users?"

Your answer framework

Start with the problem, then solve layer by layer:

"The current system is synchronous — each query blocks while waiting for Pinecone and OpenAI responses. The first fix is converting to async FastAPI, which lets one server handle 15+ concurrent requests without adding any infrastructure.

For horizontal scaling, I containerise the app with Docker and deploy multiple instances on AWS ECS behind an Application Load Balancer. Because the app is stateless — documents in S3, embeddings in Pinecone — any instance handles any request.

For cost efficiency, I add a query cache. At 2000 DAUs, a 30% cache hit rate saves roughly $108/month in LLM costs. I use Redis via AWS ElastiCache, shared across all ECS instances.

For observability, every request generates a Langfuse trace and infrastructure health goes to CloudWatch."

7.2 "How do you think about cost in this system?"

Your answer

"At 2000 DAUs averaging 10 queries each, that's 600,000 LLM calls/month. At GPT-4o mini rates, uncached that's ~$360/month.

I address this three ways: first, a query cache at 30% hit rate brings costs to ~$252/month. Second, I route simple queries to lighter models — not every question needs the most expensive model. Third, I batch embedding calls during ingestion rather than one-by-one.

I track per-query cost in Langfuse. If the average spikes, I know immediately and can investigate which queries are unusually expensive."

7.3 "How do you know your system is giving good answers?"

Your answer

"I have observability at two levels.

For performance: Langfuse traces every query — retrieval quality, exact prompt sent, response, latency per step, cost per call.

For quality: I run RAGAS evaluation weekly on a 15-question test set. I track faithfulness, answer relevancy, context precision, and context recall over time. If faithfulness drops after a code change, I know the change degraded quality.

[Open Langfuse dashboard] — here's a real trace from a query I ran earlier."

Quick Reference — All Concepts at a Glance

Term	One-line definition
Horizontal scaling	Add more servers rather than bigger servers
Vertical scaling	Make the existing server bigger (CPU, RAM)
Stateless design	Server doesn't remember previous requests; state lives in shared storage
Load balancer	Distributes incoming traffic across multiple server instances
async / await	Non-blocking code — server handles other requests while waiting for I/O
Cache hit / miss	Hit = answer found in cache (fast, free). Miss = must compute answer.
Semantic caching	Cache based on meaning, not exact string — catches similar questions
TTL	Expiry time on cached items. Prevents stale data.
Message queue	Buffer for slow background tasks (e.g. document ingestion)
Connection pooling	Reuse pre-opened DB connections instead of opening new ones per request
Read replica	Copy of database that handles read queries, reducing load on primary
Rate limiting	Cap requests per user/IP per minute to prevent abuse and cost overruns
Trace (Langfuse)	Complete record of one request's journey through the system
Faithfulness (RAGAS)	How grounded the answer is in retrieved source material. Measures hallucination.
Context Precision (RAGAS)	How relevant the retrieved chunks were to the question asked
API Gateway	Front door to your service — handles auth, rate limiting, SSL
ECS	AWS service that runs and scales your Docker containers
ECR	AWS storage for your Docker images
S3	AWS infinite file storage — for PDFs, images, model artifacts
CloudWatch	AWS monitoring — logs, metrics, alarms for infrastructure health
AWS Bedrock	AWS-managed access to LLMs (Claude, Llama, Mistral) via one unified API
ALB (App Load Balancer)	AWS load balancer that distributes HTTP traffic across ECS instances
Fargate	AWS serverless compute for ECS — no EC2 to manage

Good luck on April 14th.

Every concept in these notes maps directly to DocMind and to the interview. You are not memorising definitions — you are building a mental model of a real system. When you write async def, you are preventing the chai stall from breaking. When you open Langfuse, you are a production engineer who knows exactly what their system is doing.